# ISLS Extension Architecture v1.0.0

Manifest Layer, Registry Infrastructure,
Capsule Protocol, Spiral Scheduler, and Execute Mode

A Complementary Architecture for the
Intelligent Semantic Ledger Substrate

Sebastian Klemm

16 March 2026

**Abstract**

This document specifies four extension crates and one engine modification for the Intelligent Semantic Ledger Substrate (ISLS). Together they complete the PSP (Post-Symbolic Programming) substrate by adding: (1) a run-level Execution Manifest binding all artifacts of a complete run into a single content-addressed meta-artifact; (2) a formal digest-bound Registry infrastructure for operators, profiles, and obligations; (3) a generic evidence-bound Capsule protocol (OLP) enabling policy-controlled secret release gated by deterministic run evidence; (4) a Spiral Scheduler providing adaptive tick granularity; and (5) an Execute mode enabling generative program execution alongside the existing analytical discovery mode.

All extensions preserve the 20 structural invariants (I1–I20), all 130+ acceptance tests, deterministic replay, and append-only semantics of the ISLS core (C1–C11). The specification is implementation-ready for autonomous coding agents.

**Status:** Implementation Specification.
**Parent:** ISLS v1.0.0 Canonical Master Specification.
**Target:** Rust workspace under `isls/crates/`.

# Contents


**I Foundations** **5**

**1 Purpose and Scope** **5**
1.1 Purpose . . . . . 5
1.2 Scope . . . . . 5
1.3 Relationship to Existing Crates . . . . . 5
1.4 Invariant Preservation . . . . . 5
1.5 Technical Constraints . . . . . 6

**II Crate C12: `isls-registry`** **6**

**2 Purpose** **6**

**3 Data Model** **6**

**4 Operator Registry** **7**

**5 Profile Registry** **7**

**6 Obligation Registry** **7**

**7 Rust API** **7**

**8 Acceptance Tests** **8**

**III Crate C13: `isls-manifest`** **9**

**9 Purpose** **9**

**10 Data Model** **9**

**11 Construction Algorithm** **10**

**12 Verification** **10**

**13 Rust API** **10**

**14 Acceptance Tests** **12**

**IV Crate C14: `isls-capsule`** **12**

**15 Purpose** **12**

**16 Threat Model** **12**

**17 Core Objects** **12**

**18 Key Schedule** **13**

**19 Protocol Procedures** **13**
19.1 Seal . . . . . 13
19.2 Open . . . . . 14

**20 Rust API** **14**

**21 Acceptance Tests** **15**

**V Crate C15: `isls-scheduler`** **15**

**22 Purpose** **15**

**23 Formal Model** **16**

**24 Configuration** **16**

**25 Rust API** **16**

**26 Acceptance Tests** **17**

**VI Engine Extension: Execute Mode** **17**

**27 Purpose** **17**

**28 Formal Model** **17**

**29 CLI Integration** **18**

**30 Rust API** **18**

**31 The Discovery-Execute Loop** **19**

**VII Dependency Graph and Build Order** **19**

**32 Complete Dependency Graph** **19**

**33 Build Order** **20**

**34 External Dependencies** **20**

**VIII Implementation Handoff** **20**

**35 Normative Data Types** **20**

**36 Test Obligations** **21**

**37 Verification Sequence** **21**

**38 Completion Criteria** **21**

**A Invariant Cross-Reference** **22**

**B Symbol Additions** **22**

# Part I
# Foundations

## 1 Purpose and Scope

### 1.1 Purpose

This specification defines the minimal set of extensions required to evolve ISLS from a domain-specific analytical engine into a *universal PSP substrate* capable of three operational manifestations:

(a) **Analytical** (existing): discover programs latent in observed data and crystallize them.

(b) **Generative** (new): accept an operator graph or crystal as input and execute it through the staged closure, producing new artifacts.

(c) **Cryptographic** (new): bind secrets to evidence certificates produced by analytical or generative runs, releasing them only upon verified policy satisfaction.

All three manifestations share: the same state space $\Sigma_0 = \mathbb{R}^5$, the same canonicalization (JCS + SHA-256), the same Run Descriptor, the same Evidence Chain, the same Crystal structure, and the same gate cascade.

### 1.2 Scope

- **In scope:** Four new crates (`isls-manifest`, `isls-registry`, `isls-capsule`, `isls-scheduler`) and two engine extensions (Spiral Scheduler integration, Execute mode).
- **Out of scope:** Language frontends (Sanskroot or other surface syntaxes), domain-specific operator implementations, cryptographic primitive design, external KMS/HSM integration details.

### 1.3 Relationship to Existing Crates

| Existing | New | Relationship |
|---|---|---|
| `isls-types` | `isls-registry` | Registry types extend `isls-types`; all registry entries use existing `content_address`. |
| `isls-archive` | `isls-manifest` | Manifest reads the crystal archive and produces a run-level meta-artifact. |
| `isls-consensus` | `isls-capsule` | Capsule policies reference gate evidence produced by `isls-consensus`. |
| `isls-engine` | `isls-scheduler` | Scheduler wraps the macro-step loop; engine gains `execute` mode. |

### 1.4 Invariant Preservation

**Requirement 1.1** (Extension Non-Regression)**.** All extensions **MUST** preserve invariants I1–I20 of the ISLS v1.0.0 specification. All existing acceptance tests (AT-01 through AT-20) and all existing unit/integration tests **MUST** continue to pass without modification.

### 1.5 Technical Constraints

**Requirement 1.2** (Workspace Discipline)**.** All new crates **MUST** adhere to the established workspace discipline:

- No `unsafe` Rust code.
- No `rand` dependency; determinism via `RunDescriptor`.
- `BTreeMap` for all deterministic-order collections.
- JCS (RFC 8785) canonical serialization via `serde_jcs`.
- SHA-256 content addressing via `sha2`.
- Append-only semantics; no `delete` methods.
- All operators version-pinned.

# Part II
# Crate C12: `isls-registry`

## 2 Purpose

The registry crate provides a formal, digest-bound catalog system for operators, profiles, and obligations. It replaces the flat `operator_versions: BTreeMap<String, String>` in existing crystals with a structured, content-addressed registry infrastructure that satisfies the PCR registry requirements.

## 3 Data Model

**Definition 3.1** (Registry Entry)**.** A registry entry is a tuple

$$e = \langle \texttt{id}, \texttt{name}, \texttt{version}, \texttt{digest}, \texttt{kind}, \texttt{metadata} \rangle$$

where:

- `id`: content-address $\mathrm{Dig}(\mathrm{Canon}(e_{\mathrm{core}}))$,
- `name`: human-readable identifier (e.g., `"BandOperator"`),
- `version`: semantic version string (e.g., `"1.0.0"`),
- `digest`: SHA-256 of the canonical serialization of the entry's operational content,
- `kind` $\in \{\texttt{Operator}, \texttt{Profile}, \texttt{Obligation}, \texttt{Macro}\}$,
- `metadata`: `BTreeMap<String, String>` for extensible key-value annotations.

**Definition 3.2** (Registry)**.** A registry $\mathcal{R}$ is an append-only, content-addressed collection of registry entries:

$$\mathcal{R} = \langle \texttt{registry_id}, \texttt{kind}, \texttt{entries}, \texttt{digest} \rangle$$

where $\texttt{registry_id} = \mathrm{Dig}(\mathrm{Canon}(\texttt{entries}))$ and `digest` is the SHA-256 of the entire canonical registry serialization.

**Definition 3.3** (Registry Set)**.** A registry set is a tuple of four registries:

$$\mathcal{R}_{\mathrm{all}} = (\mathcal{R}_{\mathrm{op}}, \mathcal{R}_{\mathrm{prof}}, \mathcal{R}_{\mathrm{obl}}, \mathcal{R}_{\mathrm{macro}})$$

bound in the Run Descriptor by their respective digests.

# 4 Operator Registry

**Definition 4.1** (Operator Registration)**.** Each operator in $\Omega_{\mathsf{ISLS}}$ **MUST** be registered with:

- `name`: canonical operator name,
- `version`: semantic version,
- `type_signature`: input type → output type,
- `role`: one of {`ingest`, `canon`, `measure`, `solve`, `morph`, `gate`, `select`, `merge`, `emit`} (PSP kernel roles),
- `determinism_certificate`: boolean assertion that the operator is deterministic under fixed $(\Sigma, \mathrm{RD})$,
- `digest`: SHA-256 of the canonical serialization of the above fields.

**Requirement 4.1** (Registry Binding in RD)**.** The Run Descriptor **MUST** contain the digests of all four registries. Any operator, profile, obligation, or macro used during a run **MUST** be resolvable from the registry referenced in the RD. An operator not present in the registry **MUST NOT** be executed.

**Requirement 4.2** (Drift Detection)**.** If an operator's implementation digest does not match its registry entry digest, the system **MUST** raise an operator-drift fault (Invariant I20). This check **MUST** occur before operator execution, not after.

# 5 Profile Registry

**Definition 5.1** (Profile Entry)**.** A profile entry binds a named configuration to a specific parameter set:

- `name`: one of {`Minimal`, `Standard`, `HighRigor`} or custom,
- `thresholds`: all gate threshold values,
- `normalization`: all normalization parameters $(\mu_D, \mu_Q, \lambda, \mu_r, \ldots)$,
- `carrier`: carrier geometry configuration,
- `consensus`: dual consensus parameters,
- `digest`: SHA-256 of canonical serialization.

# 6 Obligation Registry

**Definition 6.1** (Obligation Entry)**.** An obligation entry defines a proof requirement:

- `name`: obligation identifier (e.g., `"evidence_chain_complete"`),
- `gate_ids`: which gates must produce evidence,
- `evidence_schema`: canonical JSON schema of required evidence fields,
- `verification_procedure`: reference to verifier function name and version,
- `digest`: SHA-256 of canonical serialization.

# 7 Rust API

```
// isls-registry/src/lib.rs

pub struct RegistryEntry {
    pub id: Hash256,
    pub name: String,
    pub version: String,
```

```
    pub digest: Hash256,
    pub kind: RegistryKind,
    pub metadata: BTreeMap<String, String>,
}

pub enum RegistryKind { Operator, Profile, Obligation, Macro }

pub struct Registry {
    pub registry_id: Hash256,
    pub kind: RegistryKind,
    pub entries: BTreeMap<String, RegistryEntry>,  // name -> entry
    pub digest: Hash256,
}

pub struct RegistrySet {
    pub operators: Registry,
    pub profiles: Registry,
    pub obligations: Registry,
    pub macros: Registry,
}

impl Registry {
    pub fn new(kind: RegistryKind) -> Self;
    pub fn register(&mut self, entry: RegistryEntry) -> Result<()>;
    pub fn resolve(&self, name: &str) -> Option<&RegistryEntry>;
    pub fn verify_digest(&self, name: &str, expected: &Hash256) -> bool;
    pub fn compute_digest(&self) -> Hash256;
}

impl RegistrySet {
    pub fn from_config(config: &Config) -> Self;
    pub fn digests(&self) -> BTreeMap<String, Hash256>;
    pub fn verify_operator(&self, name: &str, digest: &Hash256) -> bool;
}
```

# 8 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-R1 | Registry content address | Register an operator; verify `id` = SHA-256(JCS(core fields)). |
| AT-R2 | Drift detection | Modify operator implementation; verify drift fault raised before execution. |
| AT-R3 | RD binding | Create RD with registry digests; verify run rejects unregistered operator. |
| AT-R4 | Append-only | Attempt to remove registry entry; verify rejection. |
| AT-R5 | Deterministic digest | Register same entries in different order; verify identical registry digest (BTreeMap ordering). |

# Part III
# Crate C13: `isls-manifest`

## 9 Purpose

The manifest crate produces a single content-addressed meta-artifact that binds an entire run—all crystals, all traces, all evidence, all registry references—into one verifiable object. This satisfies the PCR Execution Manifest requirement and enables offline replay without external queries.

## 10 Data Model

**Definition 10.1** (Execution Manifest)**.** An execution manifest is a tuple

$$M = \langle \texttt{run_id}, \texttt{rd_digest}, \texttt{program_id}, \texttt{input_digest}, \texttt{crystal_digests}, \texttt{trace_digests}, \texttt{registry_digests}, \texttt{me}$$

where:

- $\texttt{run_id} = \mathrm{Dig}(\mathrm{Canon}(M_{\mathrm{core}}))$: content address of the manifest,
- `rd_digest`: SHA-256 of the canonical Run Descriptor,
- `program_id`: identifier of the program executed (crystal ID for execute mode, `"discovery"` for analytical mode),
- `input_digest`: SHA-256 of the canonical observation log or input program,
- `crystal_digests`: ordered list of all crystal IDs produced during the run,
- `trace_digests`: ordered list of per-tick trace digests,
- `registry_digests`: map of registry kind $\rightarrow$ digest,
- `mef_head`: head digest of the MEF evidence chain (optional),
- `timestamp`: extrinsic time of manifest creation.

**Definition 10.2** (Trace Entry)**.** A trace entry records one macro-step:

$$T_k = \langle k, \texttt{input_digest}_k, \texttt{state_digest}_k, \texttt{crystal_id}_k, \texttt{gate_snapshot}_k, \texttt{metrics_digest}_k \rangle$$

where $\texttt{crystal_id}_k$ is `None` if no crystal was committed at step $k$.

**Definition 10.3** (Replay Pack)**.** A replay pack is a self-contained bundle sufficient for offline replay:

$$\mathrm{RP} = \langle M, \mathrm{RD}, \texttt{observation_log}, \mathcal{R}_{\mathrm{all}}, \texttt{boundary_evidence} \rangle$$

A replay pack **MUST** enable deterministic reproduction of the run without any external queries.

# 11 Construction Algorithm

**Algorithm 1** BuildManifest: Construct execution manifest post-run

**Require:** Completed run state $S_K$, Run Descriptor RD, trace entries $\{T_0, \ldots, T_{K-1}\}$, archive $A$, registry set $\mathcal{R}_{\text{all}}$

**Ensure:** Execution manifest $M$

```
rd_digest ← Dig(Canon(RD))
input_digest ← Dig(Canon(observation_log))
crystal_digests ← [c.crystal_id | c ∈ A.crystals]
trace_digests ← [Dig(Canon(T_k)) | k = 0, ..., K − 1]
registry_digests ← R_all.digests()
mef_head ← A.evidence_chain_head()
M_core ← (rd_digest, input_digest, crystal_digests, trace_digests, registry_digests, mef_head)

run_id ← Dig(Canon(M_core))
M ← (run_id, M_core, timestamp)
return M
```

# 12 Verification

**Definition 12.1** (Manifest Verification)**.** A manifest verifier **MUST** check, in order:

(MV1) `run_id` equals $\text{Dig}(\text{Canon}(M_{\text{core}}))$.
(MV2) `rd_digest` matches the provided RD.
(MV3) Every entry in `crystal_digests` resolves in the archive.
(MV4) Every trace digest is recomputable from the corresponding trace entry.
(MV5) Registry digests match the provided registries.
(MV6) If `mef_head` is present, the evidence chain is verifiable from head to genesis.

On first divergence, the verifier **MUST** report which check failed and at which index.

# 13 Rust API

```rust
// isls-manifest/src/lib.rs

pub struct ExecutionManifest {
    pub run_id: Hash256,
    pub rd_digest: Hash256,
    pub program_id: String,
    pub input_digest: Hash256,
    pub crystal_digests: Vec<Hash256>,
    pub trace_digests: Vec<Hash256>,
    pub registry_digests: BTreeMap<String, Hash256>,
    pub mef_head: Option<Hash256>,
    pub timestamp: f64,
}

pub struct TraceEntry {
    pub tick: u64,
```

```
    pub input_digest: Hash256,
    pub state_digest: Hash256,
    pub crystal_id: Option<Hash256>,
    pub gate_snapshot: GateSnapshot,
    pub metrics_digest: Hash256,
}

pub struct ReplayPack {
    pub manifest: ExecutionManifest,
    pub rd: RunDescriptor,
    pub observation_log: Vec<Vec<Vec<u8>>>,
    pub registries: RegistrySet,
    pub boundary_evidence: Vec<EvidenceEntry>,
}

pub fn build_manifest(
    rd: &RunDescriptor,
    traces: &[TraceEntry],
    archive: &Archive,
    registries: &RegistrySet,
    program_id: &str,
    observation_log: &[Vec<Vec<u8>>],
) -> ExecutionManifest;

pub fn verify_manifest(
    manifest: &ExecutionManifest,
    rd: &RunDescriptor,
    archive: &Archive,
    traces: &[TraceEntry],
    registries: &RegistrySet,
) -> Result<(), ManifestError>;

pub fn build_replay_pack(
    manifest: ExecutionManifest,
    rd: RunDescriptor,
    observation_log: Vec<Vec<Vec<u8>>>,
    registries: RegistrySet,
    boundary_evidence: Vec<EvidenceEntry>,
) -> ReplayPack;
```

## 14 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-M1 | Manifest content address | Build manifest; verify `run_id` = SHA-256(JCS(core)). |
| AT-M2 | Manifest verification | Build and verify manifest; all MV1–MV6 pass. |
| AT-M3 | Tamper detection | Alter one crystal digest; verify verification fails at MV3. |
| AT-M4 | Replay pack sufficiency | Build replay pack; replay from pack alone; verify identical crystals. |
| AT-M5 | Trace determinism | Run twice with same RD; verify identical trace digests. |

# Part IV
# Crate C14: `isls-capsule`

## 15 Purpose

The capsule crate implements the Operator Lock Protocol (OLP): a generic evidence-bound access control mechanism in which secrets are released only upon verified satisfaction of a deterministic program's gate obligations. The capsule is domain-agnostic; any ISLS program (analytical or generative) can serve as a lock.

## 16 Threat Model

**Definition 16.1** (Adversary Capabilities)**.** An adversary may:
(i) read the lock program $P_L$, capsule headers, policies, and manifests,
(ii) copy, replay, and attempt to tamper with bundles and capsules,
(iii) run the verifier and the lock program arbitrarily often,
(iv) attempt offline attacks on ciphertexts.

**Definition 16.2** (Security Objectives)**.** OLP provides:
(i) **Confidentiality**: secrets remain confidential without access to cryptographic key material.
(ii) **Policy-bound release**: `open` succeeds only if lock obligations pass and evidence verifies.
(iii) **Tamper evidence**: any modification to capsule, bundle, or manifest is detected.
(iv) **Replay stability**: verification results are invariant under replay (deterministic).

## 17 Core Objects

**Definition 17.1** (Lock)**.** A lock is a tuple

$$L = (P_L, \Gamma, \mathrm{RD}_L)$$

where $P_L$ is a public operator program (constraint program or HDAG), $\Gamma$ is the set of gate obligations and thresholds, and $\mathrm{RD}_L$ is the RD profile binding canonicalization, hash algorithms, tie-break rules, and registry digests.

**Definition 17.2** (Capsule)**.** A capsule is an encrypted artifact:

$$C = \langle \texttt{schema_version}, \texttt{alg}, \texttt{nonce}, \texttt{ciphertext}, \texttt{aad}, \texttt{policy}, \texttt{bind} \rangle$$

where:

- `alg`: AEAD algorithm identifier (default: `AES-256-GCM`),
- `nonce`: unique per capsule,
- `ciphertext`: AEAD.Enc($K_S$, `nonce`, `plaintext`, `aad`),
- `aad`: canonical JSON of policy and bind fields (authenticated but not encrypted),
- `policy`: capsule policy $\Pi_C$,
- `bind`: digests binding the capsule to PSP objects.

**Definition 17.3** (Capsule Policy)**.** A capsule policy $\Pi_C$ constrains admissible opening contexts:

- `require_lock_program_id`: digest of $P_L$,
- `require_rd_digest`: digest of $\mathrm{RD}_L$,
- `require_gate_proofs`: list of required gate evidence item digests,
- `require_manifest_id`: optional; digest of required execution manifest,
- `expires_at`: optional; UTC timestamp after which opening is forbidden,
- `max_uses`: optional; maximum number of successful opens.

# 18 Key Schedule

**Definition 18.1** (Session Key Derivation)**.** Given a master key $K_M$ (external, e.g., KMS/HSM/local keychain), define the binding string:

$$B = \mathrm{Dig}(\mathrm{RD}_L) \parallel \texttt{run_id} \parallel P_L^{\mathrm{id}} \parallel T^{\mathrm{dig}} \parallel \mathrm{RP}^{\mathrm{dig}}$$

where values are taken from the execution manifest. Then derive:

$$K_S = \mathrm{HKDF}(K_M,\ \mathrm{salt} = \mathrm{Dig}(\mathrm{Canon}(\Sigma_0)),\ \mathrm{info} = B)$$

**Requirement 18.1** (Binding Minimality)**.** $B$ **MUST** include at least $\mathrm{Dig}(\mathrm{RD}_L)$ and `run_id`. If boundary operators are used, the replay pack digest **MUST** be included to prevent opening under missing boundary evidence.

# 19 Protocol Procedures

## 19.1 Seal

**Algorithm 2** SEAL: Encrypt secret under evidence-bound policy

**Require:** Secret $S$, lock $L = (P_L, \Gamma, \mathrm{RD}_L)$, master key $K_M$
**Ensure:** Capsule $C$

1: Execute $P_L$ on $\Sigma_0$ under $\mathrm{RD}_L$, producing manifest $M$, trace $T$, evidence, and replay pack RP
2: Verify all gate obligations $\Gamma$ pass with evidence
3: Derive $K_S$ from $K_M$ and bindings $B$ (definition 18.1)
4: `nonce` $\leftarrow$ deterministic nonce from Dig(`run_id`$\|$`seal_counter`)
5: `aad` $\leftarrow$ Canon($\Pi_C\|$`bind`)
6: `ciphertext` $\leftarrow$ AEAD.Enc($K_S$, `nonce`, $S$, `aad`)
7: $C \leftarrow$ (`schema_version`, `alg`, `nonce`, `ciphertext`, `aad`, $\Pi_C$, `bind`)
8: **return** $C$

### 19.2 Open

**Algorithm 3** OPEN: Decrypt secret under verified evidence

**Require:** Capsule $C$, lock $L$, manifest $M$, master key $K_M$
**Ensure:** Secret $S$ or $\perp$

```
Verify manifest M via MV1–MV6 (definition 12.1)
Check policy Π_C: program ID, RD digest, gate proofs
If expires_at set and current time exceeds it:
return ⊥
If max_uses set and exceeded:
return ⊥
Verify boundary replay sufficiency if boundary IO was used
Derive K_S using same derivation rule (definition 18.1)
S ← AEAD.Dec(K_S, nonce, ciphertext, aad)
If decryption fails:
return ⊥
return S
```

**Invariant 19.1** (Capsule Integrity)**.** Any modification to the capsule's `aad`, `policy`, or `bind` fields **MUST** cause AEAD .Dec to fail, because these fields are authenticated as associated data.

## 20 Rust API

```
// isls-capsule/src/lib.rs

pub struct Lock {
    pub program: ConstraintProgram,  // or HDAG
    pub obligations: Vec<String>,    // obligation registry names
    pub rd: RunDescriptor,
}

pub struct CapsulePolicy {
    pub require_lock_program_id: Hash256,
    pub require_rd_digest: Hash256,
    pub require_gate_proofs: Vec<Hash256>,
    pub require_manifest_id: Option<Hash256>,
    pub expires_at: Option<f64>,
    pub max_uses: Option<u64>,
}

pub struct Capsule {
    pub schema_version: String,
    pub alg: String,
    pub nonce: Vec<u8>,
    pub ciphertext: Vec<u8>,
    pub aad: Vec<u8>,
    pub policy: CapsulePolicy,
    pub bind: BTreeMap<String, Hash256>,
```

```
}

pub fn seal(
    secret: &[u8],
    lock: &Lock,
    master_key: &[u8; 32],
    manifest: &ExecutionManifest,
) -> Result<Capsule, CapsuleError>;

pub fn open(
    capsule: &Capsule,
    lock: &Lock,
    master_key: &[u8; 32],
    manifest: &ExecutionManifest,
) -> Result<Vec<u8>, CapsuleError>;

pub fn verify_policy(
    capsule: &Capsule,
    manifest: &ExecutionManifest,
) -> Result<(), PolicyError>;
```

## 21 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-C1 | Seal-open round-trip | Seal a secret; open with valid manifest; verify identical plaintext. |
| AT-C2 | Policy rejection | Open with manifest missing a required gate proof; verify rejection. |
| AT-C3 | Tamper detection | Alter capsule AAD; verify AEAD decryption fails. |
| AT-C4 | Expiry enforcement | Set `expires_at` in the past; verify open returns $\perp$. |
| AT-C5 | Replay stability | Seal and open twice with same inputs; verify identical results. |
| AT-C6 | Wrong manifest | Open with manifest from a different run; verify rejection. |

# Part V

# Crate C15: `isls-scheduler`

## 22 Purpose

The scheduler crate provides adaptive tick granularity based on system dynamics. Instead of flat ticks of uniform duration, the Spiral Scheduler observes metric behavior and adjusts tick frequency: finer granularity during high deformation/friction (turbulence), coarser during stability.

# 23 Formal Model

**Definition 23.1** (Spiral Segmentation)**.** A spiral segmentation maps macro-step index $k$ to an effective tick count $n_k \in \mathbb{N}_{>0}$ through a scheduling function:

$$n_k = \sigma(d_k, F_k^*, S_k^*, \theta_{\text{sched}})$$

where $d_k$ is the deformation metric, $F_k^*$ is friction, $S_k^*$ is shock, and $\theta_{\text{sched}}$ collects scheduling parameters.

**Definition 23.2** (Scheduling Function)**.** The canonical scheduling function is:

$$\sigma(d, F, S, \theta) = \text{clamp}(\lfloor n_{\min} + (n_{\max} - n_{\min}) \cdot \max(d, F, S) \rfloor\,,\ n_{\min},\ n_{\max})$$

where $n_{\min} \geq 1$ is the minimum sub-steps per macro-step and $n_{\max}$ is the maximum.

**Definition 23.3** (Sub-Step)**.** A sub-step executes the intrinsic dynamics $\frac{dH}{dt_2} = \Phi(H)$ with effective $\Delta t_2' = \Delta t_2 / n_k$, updating carrier phase, embeddings, and resonance state $n_k$ times before the extrinsic commit decision is evaluated.

**Invariant 23.1** (Extrinsic Discreteness)**.** The extrinsic commit index $k$ increments exactly once per macro-step regardless of $n_k$. Sub-steps are intrinsic-time refinements only. They **MUST NOT** produce commits, crystals, or append to the archive. Only the final composite state after all $n_k$ sub-steps enters the extrinsic decision pipeline (extraction $\rightarrow$ consensus $\rightarrow$ commit).

**Invariant 23.2** (Deterministic Scheduling)**.** The scheduling function $\sigma$ **MUST** be deterministic under fixed $(\Sigma, \text{RD})$. The scheduling parameters $\theta_{\text{sched}}$ **MUST** be part of the Run Descriptor.

# 24 Configuration

```
scheduler:
  enabled: true
  n_min: 1
  n_max: 10
  strategy: "max_pressure"  # or "weighted", "fixed"
```

When `enabled: false`, the scheduler degenerates to $n_k = 1$ for all $k$ (flat ticks, backward-compatible).

# 25 Rust API

```
// isls-scheduler/src/lib.rs

pub struct SchedulerConfig {
    pub enabled: bool,
    pub n_min: u32,
    pub n_max: u32,
    pub strategy: ScheduleStrategy,
}

pub enum ScheduleStrategy {
    MaxPressure,   // n_k = clamp(n_min + (n_max-n_min)*max(d,F,S))
```

```
    Weighted { w_d: f64, w_f: f64, w_s: f64 },
    Fixed(u32),    // constant sub-steps
}

pub fn compute_substeps(
    d: f64, f_friction: f64, s_shock: f64,
    config: &SchedulerConfig,
) -> u32;

pub fn run_substeps(
    state: &mut GlobalState,
    n_substeps: u32,
    config: &Config,
);
```

## 26 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-S1 | Disabled passthrough | Set `enabled: false`; verify $n_k = 1$ for all $k$. |
| AT-S2 | Adaptive scaling | Inject high-deformation data; verify $n_k > 1$. Inject low-deformation; verify $n_k = n_{\min}$. |
| AT-S3 | Determinism | Run twice with same RD; verify identical $n_k$ sequences. |
| AT-S4 | Extrinsic invariance | Verify commit index increments exactly once per macro-step regardless of $n_k$. |
| AT-S5 | Backward compatibility | Run full S-Basic scenario with scheduler disabled; verify identical results to current behavior. |

# Part VI
# Engine Extension: Execute Mode

## 27 Purpose

The execute mode adds a generative operational manifestation to ISLS. In the existing analytical mode, the engine ingests data and discovers programs. In execute mode, the engine accepts a program (crystal, constraint program, or HDAG) as input and executes it through the staged closure, producing new artifacts.

## 28 Formal Model

**Definition 28.1** (Execute Input)**.** An execute input is one of:
(a) A `SemanticCrystal` (the `constraint_program` field is executed),
(b) A `ConstraintProgram` (directly),
(c) An HDAG (a `PersistentGraph` with operator-typed edges).

**Definition 28.2** (Execute Semantics)**.** Given execute input $P$ and initial state $\Sigma_0$ (which may

be empty or pre-populated), the execute mode runs the staged closure:

$$\Sigma_K = P(\Sigma_0) \quad \text{under RD}$$

producing: a final state $\Sigma_K$, zero or more new crystals, a complete trace, and an execution manifest.

**Definition 28.3** (Staged Closure for Execute Mode)**.** The staged closure in execute mode follows the PCR stages:

(S1) **Ingest**: Canonicalize input program $P$.
(S2) **Canon**: Canonicalize initial state $\Sigma_0$.
(S3) **Expand**: Apply macro expansions from the macro registry (if $P$ contains macro references).
(S4) **Measure/Embed**: Compute 5D embeddings of the resulting graph structure.
(S5) **Solve**: Execute operators in topological order.
(S6) **Gate**: Evaluate all gate obligations (Kairos cascade).
(S7) **Coagula/Select**: If gates pass, crystallize.
(S8) **Emit**: Emit crystals, trace, and manifest.

**Requirement 28.1** (Execute-Discover Symmetry)**.** The execute mode and the discovery mode **MUST** share:

- the same crystal format,
- the same evidence chain structure,
- the same manifest format,
- the same gate cascade,
- the same determinism contract.

A crystal produced by execute mode **MUST** be indistinguishable in format from a crystal produced by discovery mode. The `program_id` field in the manifest distinguishes the provenance.

## 29 CLI Integration

```
# Execute a constraint program from a crystal file
isls execute --input crystal.json --ticks 50

# Execute an HDAG from a graph file
isls execute --input program.hdag.json --ticks 100 --output results/

# Execute and seal result as a capsule
isls execute --input crystal.json --seal --lock-policy policy.json
```

## 30 Rust API

```
// In isls-engine/src/lib.rs (extension)

pub enum ExecuteInput {
    Crystal(SemanticCrystal),
    Program(ConstraintProgram),
    Hdag(PersistentGraph),
}
```

```
pub fn execute(
    input: ExecuteInput,
    initial_state: Option<GlobalState>,
    config: &Config,
    registries: &RegistrySet,
    ticks: usize,
) -> Result<(Vec<Option<SemanticCrystal>>, ExecutionManifest)>;
```

## 31 The Discovery-Execute Loop

The complete PSP substrate enables a self-reinforcing cycle:

1. **Discover**: Ingest data, run analytical mode, produce crystals encoding discovered programs.
2. **Execute**: Take a discovered crystal, run it in execute mode on new data, produce new artifacts.
3. **Observe**: Ingest the execution artifacts as new observations.
4. **Re-discover**: Find higher-order patterns in execution results.
5. **Crystallize**: Produce meta-crystals encoding patterns of patterns.

Each cycle produces manifests and evidence chains that are independently verifiable. The entire history is append-only, content-addressed, and deterministically replayable.

**Theorem 31.1** (Loop Determinism)**.** *If each iteration of the discovery-execute loop uses a fixed* RD *with fixed registry digests, and all inputs are canonical, then the entire multi-iteration loop is deterministic: identical initial observations produce identical meta-crystals at every nesting depth.*

*Proof.* By induction. Each individual run (discover or execute) is deterministic by Theorem 23.1 of the ISLS v1.0.0 specification. The output of one run is canonicalized before serving as input to the next. Since canonicalization is idempotent and all operators are deterministic, the composition is deterministic at every depth. □

# Part VII
# Dependency Graph and Build Order

## 32 Complete Dependency Graph

```
isls-types       <-- (all crates)
    ^
isls-registry    <-- (all crates that use operators)
    ^
isls-observe <-- isls-persist <-- isls-extract
                      ^                ^
              isls-consensus <-- isls-carrier
                      ^
              isls-archive <-- isls-morph
                      ^
              isls-manifest  (C13: reads archive + traces)
                      ^
```

```
isls-capsule   (C14: uses manifest + archive + AEAD)
        ^
isls-scheduler (C15: wraps engine loop)
        ^
isls-engine     (extended: execute mode + scheduler)
        ^
isls-harness    (extended: manifest/capsule tests)
        ^
isls-cli        (extended: execute + seal/open commands)
```

## 33 Build Order

1. `isls-registry` (C12): no new dependencies beyond `isls-types`.
2. `isls-manifest` (C13): depends on `isls-types`, `isls-archive`, `isls-registry`.
3. `isls-capsule` (C14): depends on `isls-types`, `isls-manifest`, `isls-registry`. New external dependency: `aes-gcm` crate for AEAD.
4. `isls-scheduler` (C15): depends on `isls-types`, `isls-engine` (or shared types from it).
5. Engine extensions: modify `isls-engine` to integrate scheduler and execute mode.
6. CLI extensions: add `execute`, `seal`, `open` subcommands.
7. Harness extensions: add tests for C12–C15.

## 34 External Dependencies

| Crate | Version | Purpose |
|---|---|---|
| `aes-gcm` | 0.10 | AEAD encryption for capsules |
| `hkdf` | 0.12 | Key derivation for capsule session keys |

All other dependencies are already present in the ISLS workspace.

# Part VIII
# Implementation Handoff

## 35 Normative Data Types

```
// New types to add to isls-types/src/lib.rs

pub type Hash256 = [u8; 32];

pub struct SchedulerConfig {
    pub enabled: bool,
    pub n_min: u32,     // >= 1
    pub n_max: u32,     // >= n_min
    pub strategy: String, // "max_pressure" | "weighted" | "fixed"
    pub w_d: f64,       // weight for deformation (if weighted)
    pub w_f: f64,       // weight for friction
    pub w_s: f64,       // weight for shock
}
```

```
// Extend RunDescriptor:
pub struct RunDescriptor {
    pub config: Config,
    pub operator_versions: BTreeMap<String, String>,
    pub initial_state_digest: Hash256,
    pub seed: Option<u64>,
    // NEW:
    pub registry_digests: BTreeMap<String, Hash256>,
    pub scheduler: SchedulerConfig,
}
```

## 36 Test Obligations

Total new acceptance tests: 21 (AT-R1 through AT-R5, AT-M1 through AT-M5, AT-C1 through AT-C6, AT-S1 through AT-S5).

All existing 130+ tests **MUST** continue to pass. The final test count after extension should be $\geq 151$.

## 37 Verification Sequence

```
# Build
cargo build --workspace --release 2>&1 | grep -c warning
# MUST output: 0

# Test
cargo test --workspace -- --test-threads=1
# MUST output: >= 151 passed, 0 failed

# S-Basic with manifest
rm -rf ~/.isls
cargo run --bin isls --release -- ingest --adapter synthetic --scenario S-Basic
cargo run --bin isls --release -- run --mode shadow --ticks 100
cargo run --bin isls --release -- validate formal
# MUST output: >= 40 crystals, 100% pass rate

# Execute mode
cargo run --bin isls --release -- execute --input ~/.isls/archive/latest.json --ticks 10
# MUST produce crystals and a manifest

# Capsule round-trip
cargo run --bin isls --release -- seal --secret "test" --lock-manifest latest
cargo run --bin isls --release -- open --capsule ~/.isls/capsules/latest.json
# MUST output: "test"
```

## 38 Completion Criteria

The extension is complete when:

1. All four crates compile with zero warnings.
2. All 151+ tests pass.
3. S-Basic produces crystals with manifests.
4. Execute mode runs a discovered crystal and produces new crystals.
5. Seal/open round-trip succeeds.
6. Scheduler disabled produces identical results to pre-extension behavior.
7. Full HTML report includes manifest and capsule sections.

# A Invariant Cross-Reference

| New Invariant | Source | ISLS Base Invariant |
|---|---|---|
| Registry content-addressing | PCR §5.4 | I2 (Canonical Serialization) |
| Registry drift detection | PCR §5.4 | I20 (Operator Version Pinning) |
| Manifest content-addressing | PCR §8.1 | I2 |
| Manifest tamper-evidence | PCR §8.3 | I3 (Provenance Completeness) |
| Replay pack sufficiency | PCR §8.2 | I4 (Deterministic Replay) |
| Capsule confidentiality | OLP §5.2 | new |
| Capsule policy-bound release | OLP §5.2 | I9 (Threshold-Governed) |
| Capsule tamper-evidence | OLP §5.2 | I8 (Storage Integrity) |
| Scheduler determinism | NCTE Axiom 3 | I4 |
| Extrinsic discreteness | NCTE Axiom 9 | I10 (Commit Immutability) |
| Execute-discover symmetry | PSP §I.2 | I2, I4 |

# B Symbol Additions

| Symbol | Type | Meaning |
|---|---|---|
| $\mathcal{R}$ | Registry | Content-addressed operator/profile/obligation catalog |
| $M$ | Manifest | Execution manifest binding a complete run |
| $T_k$ | TraceEntry | Per-tick trace record |
| RP | ReplayPack | Self-contained replay bundle |
| $L$ | Lock | $(P_L, \Gamma, \mathrm{RD}_L)$ lock specification |
| $C$ | Capsule | Encrypted evidence-bound artifact |
| $\Pi_C$ | Policy | Capsule release policy |
| $K_M$ | bytes | Master key (external) |
| $K_S$ | bytes | Derived session key |
| $B$ | bytes | Binding string for key derivation |
| $\sigma(\cdot)$ | function | Spiral scheduling function |
| $n_k$ | $\mathbb{N}_{>0}$ | Sub-step count at macro-step $k$ |

Generated for ISLS v1.0.0 Extension Architecture.
Deterministic, append-only, replay-verified.